1. LL(1) Parsing

FIRST sets:

FIRST(S) = FIRST(AB) $\cup$ FIRST(eDa)

FIRST(A) = {a, c}
FIRST(B) = {d}
FIRST(C) = {e, ϵ}
FIRST(D) = {f, ϵ}
Compute step by step:

FIRST(AB) = FIRST(A) = {a, c}
FIRST(eDa) = {e}
FIRST(S) = {a, c, e}
FIRST(C) = {e, ϵ}
FIRST(D) = {f, ϵ}

FOLLOW sets:

FOLLOW(S) = { $, a }
FOLLOW(A) = {a, b, d}
FOLLOW(B) = {a}
FOLLOW(C) = {a}
FOLLOW(D) = {a}
Compute step by step:

FOLLOW(A) = FIRST(B) $\cup$ FOLLOW(S) = {d, a}

FOLLOW(B) = FIRST(C) $\cup$ FOLLOW(S) = {e, ϵ, a} - {ϵ} = {e, a}
FOLLOW(C) = FOLLOW(B) = {a}
FOLLOW(D) = FOLLOW(A) = {a}

Construct Parsing Table

Non-terminal	a	b	c	d	e	f	$
S	AB		AB		eDa		
A	ab		c				
B				dC			
C					eC, ϵ		ϵ
D						fD, ϵ	ϵ

Verify LL(1) Property

The table should have no conflicts. If conflicts exist, it's not LL(1).
The table above has conflicts for:

C on 'e' (eC, ϵ)
D on 'f' (fD, ϵ)
Hence, the grammar is not LL(1).

Apply LR(0) Parsing Method

Grammar:

P1: S → Aa | bAc | Bc | bBa

P2: A → d

P3: B → d

Augment Grammar

Add augmented rule S' → S

Construct LR(0) Items and Automaton

S' → • S

S → • Aa

S → • bAc

S → • Bc

S → • bBa

A → • d

B → • d

Construct LR(0) items:

State	Items
I0	S' → •S, S → •Aa, S → •bAc, S → •Bc, S → •bBa, A → •d, B → •d
I1	S' → S•
I2	S → b•Ac, S → b•Ba
I3	A → d•
I4	B → d•
I5	S → bA•c
I6	S → bB•a
I7	S → Aa•
I8	S → Bc•

Create LR(0) Parsing Table

State	a	b	c	d	$	S	A	B
I0		S2		S3, S4		I1	I7	I8
I1					Accept			
I2			S5				I3	I4
I3					R2			
I4					R3			
I5				S6				
I6	S7							
I7			R1					
I8					R4			
I9			R1					

2. LR(0) Parsing Method

Grammar:

P1: S → Aa | bAc | Bc | bBa

P2: A → d

P3: B → d

Step 1: Augment Grammar

Add augmented rule S' → S

Step 2: Construct LR(0) Items and Automaton

S' → • S

S → • Aa

S → • bAc

S → • Bc

S → • bBa

A → • d

B → • d

Construct LR(0) items:

State	Items
I0	S' → •S, S → •Aa, S → •bAc, S → •Bc, S → •bBa, A → •d, B → •d
I1	S' → S•
I2	S → b•Ac, S → b•Ba
I3	A → d•
I4	B → d•
I5	S → bA•c
I6	S → bB•a
I7	S → Aa•
I8	S → Bc•
I9	S → Aa•

Create LR(0) Parsing Table

State	a	b	c	d	$	S	A	B
I0		S2		S3, S4		I1	I7	I8
I1					Accept			
I2			S5				I3	I4
I3					R2			
I4					R3			
I5				S6				
I6	S7							
I7			R1					
I8					R4			
I9			R1					

3. SLR Parsing

Grammar:

P1: S → AaAb

P2: A → bA

P3: A → d

P4: S → BbBa

P5: B → d

Step 1: Augment Grammar

Add augmented rule S' → S

Step 2: Construct LR(0) Items and Automaton
Construct LR(0) items, then follow similar steps as LR(0) to create the automaton.

Step 3: Compute FOLLOW sets
FOLLOW(S) = { $, a }
FOLLOW(A) = {a, b}
FOLLOW(B) = {a}

4. LR(1) Parsing

Grammar:

P1: S → a+x+E

P2: E → a+E

P3: E → a

Step 1: Augment Grammar

Add augmented rule S' → S

Step 2: Construct LR(1) Items and Automaton

S' → • S, [$]

S → a • x+E, [$]

E → a • E, [$]

E → a • , [$]